UNIVERSITÀ DI BOLOGNA
Dipartimento di Informatica — Scienza e Ingegneria
Corso di Simulazione di Sistemi — M1 Informatica — A.A. 2025/2026

Progetto 1

Simulazione di una Coda con Vacation, N-Policy e Set-up

Bulk Arrival · Bulk Service (a,b)=(7,16) · Single Vacation Rule · OMNeT++ 6.3

Mohamed Samir Haffoudhi
mohamed.haffoudhi@studio.unibo.it
Aprile 2025

Articolo di riferimento: P. Karan, S. Pradhan — Performance Evaluation 167 (2025) 102459
https://doi.org/10.1016/j.peva.2024.102459

1. Introduzione

Questo progetto si inserisce nell'ambito del corso di **Simulazione di Sistemi** del primo anno della Laurea Magistrale in Informatica. L'obiettivo è progettare, implementare e analizzare un modello di simulazione a eventi discreti (DES) per uno specifico sistema a coda, confrontandone il comportamento al variare del carico di lavoro.

Il sistema modellato è una **coda a singolo server con servizio bulk e meccanismo di vacation**, ispirato all'articolo di riferimento:

> *P. Karan, S. Pradhan, "Analysis of a queue-length-dependent vacation queue with bulk service, N-policy, set-up time and cost optimization",* ***Performance Evaluation 167 (2025) 102459.***
> https://doi.org/10.1016/j.peva.2024.102459

Il modello implementato è una **variante semplificata** del sistema descritto nell'articolo: si considerano distribuzioni specifiche (esponenziale, uniforme discreta e continua) per i processi di arrivo, servizio, vacation e set-up, e si analizza esclusivamente la **Single Vacation Rule (SVR)**. L'aspetto principale di interesse è lo studio delle prestazioni del sistema al variare del parametro *m* (media del tempo di inter-arrivo dei batch), attraverso 20 repliche statisticamente indipendenti per ciascuna delle 4 configurazioni.

1.1 Contesto e Motivazione

I sistemi a coda con vacation (*vacation queues*) sono ampiamente utilizzati come modelli matematici in diversi contesti reali: reti di sensori wireless (WSN), sistemi di produzione manifatturiera, reti di telecomunicazione e protocolli di accesso al mezzo. In questi sistemi, il server non rimane in attesa attiva quando non ci sono abbastanza clienti da servire, ma entra in uno stato di "vacation", simulando situazioni come la modalità sleep di un relay node in una WSN, o la fase di manutenzione in un sistema di produzione.

La combinazione di **arrivi batch** (ogni evento genera un gruppo di clienti), **servizio bulk** (il server serve più clienti contemporaneamente), e **N-policy** (il server si riattiva solo quando ci sono almeno N clienti in attesa) rende il sistema particolarmente interessante dal punto di vista modellistico, introducendo non-linearità e dipendenze dal carico che la simulazione a eventi discreti permette di esplorare con precisione.

1.2 Obiettivi del Progetto

Lo studio si propone di perseguire i seguenti obiettivi specifici:

- **Implementazione:** realizzare il modello di simulazione in OMNeT++ 6.3 tramite moduli C++ custom (BatchSource, BulkServiceVacationServer), estendendo la libreria queueinglib con la logica specifica del sistema.
- **Analisi del transiente:** identificare la fase di transitorio iniziale in cui il sistema non ha ancora raggiunto il regime stazionario, e determinare un warm-up period adeguato per ciascuna configurazione.
- **Stima delle prestazioni:** calcolare le 4 misure di prestazione richieste (L, W, ρ, E[vacation]) con stime puntuali e intervalli di confidenza al 95%, via t-Student su 20 repliche.
- **Verifica di correttezza:** validare il modello attraverso la Legge di Little ($L = \lambda \cdot W$), una proprietà universale dei sistemi stazionari che funge da test di consistenza interna.
- **Analisi parametrica:** studiare come le misure di prestazione variano al variare del carico (parametro m), interpretando i risultati alla luce della teoria delle code.

2. Descrizione del Modello

Il sistema è una **coda M[X]/G(a,b)/1** con vacation e N-policy: arrivi batch poissoniani, servizio bulk batch-size-dependent, vacation queue-length-dependent (SVR), dormant state e set-up esponenziale. Di seguito si descrivono in dettaglio tutti i processi e le regole di transizione.

2.1 Processo di Arrivo (Compound Poisson)

Il processo di arrivo è un **processo di Poisson composto**: ogni evento di arrivo genera un intero batch di clienti. Il tempo tra due eventi di arrivo consecutivi è esponenziale di media *m* (parametro variabile), mentre la dimensione del batch è una variabile casuale discreta uniforme.

```
T_interarrivo ~ Exp(m)      con m ∈ {0.5, 1.0, 1.4, 2.0}
X ~ U_discreta[x1, x2] = U_discreta[1, 4]      E[X] = (1+4)/2 = 2.5
λ_batch = 1/m          λ_jobs = E[X] / m = 2.5 / m
```

Il **tasso di arrivo effettivo di job individuali** è λ_jobs = 2.5/m. Per m=0.5 si ha il massimo carico con λ_jobs=5 job/s; per m=2.0 il minimo con λ_jobs=1.25 job/s. Ogni job che arriva riceve subito un *timestamp* che permetterà di misurare il tempo di permanenza nel sistema (W).

2.2 Service Rule e Processo di Servizio

Il server adotta la **general bulk service rule (a, b)** con a=7, b=16: un batch di servizio viene avviato solo quando in coda sono presenti almeno *a* clienti (soglia minima), e vengono presi al più *b* clienti (capacità massima). Se la coda ha *r* clienti con a ≤ r ≤ b, tutti vengono serviti insieme; se r > b, si prendono i primi b e i rimanenti restano in coda. I nuovi arrivi non possono unirsi a un batch già in servizio.

Il tempo di servizio di un batch di dimensione *r* è la somma di una componente deterministica e una stocastica:

```
t_service(r) = t1 + t2
t1 = (r / b) · sp  [deterministica]   sp = 0.1,  b = 16
t2 ~ U(l, u) = U(1.0, 3.92)  [uniforme continua]
```

La componente deterministica *t1* introduce una dipendenza lineare dalla dimensione del batch: batch più grandi richiedono proporzionalmente più tempo deterministico. Il tempo medio di servizio per un batch di dimensione *r* vale:

```
E[t_service(r)] = (r/16)·0.1 + (1.0+3.92)/2 = (r/16)·0.1 + 2.46  [s]
```

Per *r* = a = 7: E[t] ≈ 2.50s; per *r* = b = 16: E[t] ≈ 2.56s. La variabilità è dominata dalla componente uniforme (range 2.92s), mentre l'effetto deterministico è modesto (variazione di 0.06s tra batch minimo e massimo).

2.3 Single Vacation Rule (SVR) con Durata Dipendente da k

Al termine di ogni servizio, il server controlla la coda. Se il numero di clienti in attesa è inferiore alla soglia minima a=7, il server entra in **vacation** (Single Vacation Rule). La **durata della vacation dipende dal numero di clienti k presenti in coda al momento dell'inizio della vacation**, con 0 ≤ k ≤ a−1 = 6:

```
t_vacation(k) = t' + t''
t' = ((k+1) / b) · sv  [deterministica]   sv = 0.2,  b = 16
t'' ~ U(z, w) = U(2.0, 3.8)  [uniforme continua]
```

Anche qui la componente deterministica introduce una dipendenza dalla lunghezza della coda al momento della vacation: più clienti ci sono in coda, più breve sarà la componente deterministica *t'* (poiché il sistema è più carico). La media della vacation varia da E[t_vac(0)] = (1/16)·0.2 + 2.9 ≈ 2.91s a E[t_vac(6)] = (7/16)·0.2 + 2.9 ≈ 2.99s. La variazione è piccola rispetto alla componente uniforme, spiegando perché E[vacation] risulta quasi costante al variare di m.

La **SVR** prevede che il server faccia *una sola vacation* al termine del servizio (a differenza della Multiple Vacation Rule, dove il server potrebbe riprendere la vacation se al ritorno la coda è ancora insufficiente). Al ritorno dalla vacation, il server controlla la N-policy.

2.4 N-Policy, Stato Dormant e Set-up

Al ritorno dalla vacation, viene applicata la **N-policy con N=20**. La condizione N ≥ b=16 è soddisfatta (20 ≥ 16), garantendo che quando il set-up termina ci siano sempre abbastanza clienti per servire un batch pieno:

- **queue ≥ N=20:** il server avvia il **set-up**, ovvero una fase di preparazione di durata esponenziale con media q=1.0s prima di iniziare il servizio.
- **queue < N=20:** il server entra in **stato dormant (IDLE)**, rimanendo in attesa passiva. Quando successivi arrivi portano la coda a N, il set-up viene avviato automaticamente.

```
t_setup ~ Exp(q)   con q = 1.0  →  E[t_setup] = 1.0 s
```

L'N-policy ha un impatto importante sulle prestazioni per bassi carichi: quando m è grande (pochi arrivi), il server può dover aspettare a lungo nello stato dormant prima che arrivino N=20 clienti. Questo aumenta il tempo di permanenza W per le configurazioni con m grande, come confermato dai risultati.

2.5 Diagramma di Transizione degli Stati

BUSY	VACATION	IDLE (dormant)	SETUP
Servizio batch ra ≤ r ≤ b	SVR, duratadipendente da k	Attesa passivaqueue < N=20	PreparazioneExp(q=1.0)

Le transizioni tra stati sono: BUSY → VACATION (queue<a dopo servizio) · BUSY → BUSY (queue≥a dopo servizio) · VACATION → SETUP (queue≥N al ritorno) · VACATION → IDLE (queue<N al ritorno) · IDLE → SETUP (queue raggiunge N) · SETUP → BUSY (setup completato).

2.6 Parametri del Modello

Parametro	Valore	Sezione omnetpp.ini	Ruolo nel modello
m	{0.5, 1.0, 1.4, 2.0}	Config m05..m20	Media inter-arrivo batch (variabile di controllo)
x1, x2	1, 4	[General]	Range batch size X ~ U_discreta[1,4], E[X]=2.5
a, b	7, 16	[General]	Min/Max clienti per batch di servizio

Parametro	Valore	Sezione omnetpp.ini	Ruolo nel modello
sp	0.1	[General]	Scala componente deterministica tempo servizio
l, u	1.0, 3.92	[General]	Limiti U(l,u) componente casuale servizio
sv	0.2	[General]	Scala componente deterministica tempo vacation
z, w	2.0, 3.8	[General]	Limiti U(z,w) componente casuale vacation
N	20	[General]	Soglia N-policy per riattivazione server
q	1.0	[General]	Media set-up time (esponenziale)
repeat	20	[General]	Run indipendenti per stima IC 95%
warmup-period	10000s	[General]	Periodo di riscaldamento (escluso dalla stima)
sim-time-limit	100000s	[General]	Durata totale di ogni run

Tabella 1 — Parametri del modello con sorgente nella configurazione OMNeT++

2.7 Condizione di Stabilità

La condizione necessaria e sufficiente per la stabilità della coda è che il fattore di utilizzo ρ sia strettamente minore di 1. Per questo sistema il fattore di utilizzo teorico si calcola come:

```
ρ = λ_jobs · E[t_service(b)] / b  <  1
```

Il caso più critico è m=0.5, con λ_jobs = 5 job/s. Il valore simulato ρ ≈ 0.864 < 1 conferma la stabilità per tutte le 4 configurazioni. La vicinanza alla saturazione per m=0.5 spiega i valori elevati di L e la maggiore variabilità osservata in quell'unica configurazione.

3. Implementazione in OMNeT++ 6.3

Il modello è implementato in **OMNeT++ 6.3** con la libreria **queueinglib**, estesa tramite due moduli C++ custom. OMNeT++ fornisce il framework per la simulazione a eventi discreti (scheduler, generatori di numeri casuali, raccolta statistica), mentre queueinglib offre primitive come `cQueue` e il modulo `Sink` per la raccolta del *lifeTime* dei job.

3.1 Architettura del Progetto

File	Ruolo e contenuto
BatchSource.h / .cc	Modulo C++ custom: generatore arrivi compound Poisson. Implementa intuniform(x1,x2) per batch size e exponential(m) per inter-arrivo.
BulkServiceVacationServer.h / .cc	Modulo C++ custom: server bulk con macchina a 4 stati (BUSY/VACATION/IDLE/SETUP), SVR queue-length-dependent, N-policy, set-up.
modello_progetto.ned	Definizione NED della rete (BatchSource→Server→Sink via IdealChannel), dichiarazione segnali @signal e statistiche @statistic.
omnetpp.ini	4 config principali (m05..m20) con repeat=20, warmup=10000s. 4 config transient (warmup=0, repeat=1) per analisi transitorio.
analisi_risultati.py	Script Python: parsing .sca/.vec, stima IC 95% via t-Student, verifica Legge di Little, boxplot e grafici, export CSV.
run_all.sh	Shell script: lancia automaticamente 80 run (4 config × 20 ripetizioni) sequenzialmente.
Makefile	Generato con opp_makemake: compila BatchSource.o e BulkServiceVacationServer.o, linka con queueinglib e OMNeT++.

Tabella 2 — Struttura del progetto

3.2 Modulo BatchSource

Il modulo `BatchSource` è un `cSimpleModule` che implementa il processo di Poisson composto tramite un self-message ricorrente (`batchTimer`). Ad ogni scatto, genera un batch e schedula il prossimo arrivo:

```
// BatchSource.cc — initialize()m  = par("m").doubleValue();   // media
inter-arrivo (parametro variabile)x1 = par("x1").intValue();     // min batch size
= 1x2 = par("x2").intValue();     // max batch size = 4timerMsg = new
cMessage("batchTimer");scheduleAt(startTime, timerMsg);   // primo arrivo a t=0//
BatchSource.cc — handleMessage()int batchSize = intuniform(x1, x2);      // X ~
U_discreta[1, 4]emit(batchSizeSignal, batchSize);for (int i = 0; i < batchSize;
i++) {    Job *job = new Job(...);    job->setTimestamp();   // marca t_arrivo per
calcolo W    send(job, "out");      // invia job singoli al
server}emit(createdSignal, batchSize);           // sum → jobs totali (per
Little)scheduleAt(simTime() + exponential(m), timerMsg);  // prossimo batch
```

I job vengono inviati **singolarmente** al server, che si occupa di accumularli internamente nella propria coda fino al raggiungimento della soglia a=7. Ogni job riceve un *timestamp* all'arrivo: la differenza tra il timestamp di uscita dal sistema (registrata dal `Sink`) e quello di arrivo costituisce il *lifeTime*, ovvero il tempo di permanenza W del singolo job.

3.3 Modulo BulkServiceVacationServer

Il modulo `BulkServiceVacationServer` implementa la macchina a stati del server tramite tre self-message (`endServiceMsg`, `endVacationMsg`, `endSetupMsg`) e una `cQueue` interna per la gestione dei clienti in attesa.

3.3.1 Formule di Servizio, Vacation e Set-up nel Codice C++

```
// Tempo servizio batch-size-dependent: t = (r/b)*sp + U(l,u)simtime_t
BulkServiceVacationServer::computeServiceTime(int r) {    double t1 = ((double)r /
(double)b) * sp;   // componente det.: (r/16)*0.1    double t2 = uniform(l, u);
// componente stoc.: U(1.0, 3.92)    return t1 + t2;}// Tempo vacation
queue-length-dependent: t = ((k+1)/b)*sv + U(z,w)simtime_t
BulkServiceVacationServer::computeVacationTime(int k) {    double t1 = ((double)(k
+ 1) / (double)b) * sv;  // (k+1)/16 * 0.2    double t2 = uniform(z, w);
// U(2.0, 3.8)    return t1 + t2;}// Set-up: esponenziale di media qsimtime_t
BulkServiceVacationServer::computeSetupTime() {    return exponential(q);   // q =
1.0}
```

3.3.2 Logica di Transizione a Fine Servizio (SVR + N-Policy)

```
// Fine servizio (endServiceMsg) — core della logica SVR + N-policyfor (auto *job
: currentBatch) {    send(job, "out");   // invia job al Sink (registra
lifeTime)}currentBatch.clear();emit(busySignal, false);batchBeingServed =
0;emit(numInSystemSignal, getSystemPopulation());  // ← CORREZIONE CRITICAint qLen
= queue.getLength();if (qLen >= a) {    tryStartService();   // abbastanza clienti
→ avvia nuovo servizio} else {    startVacation();     // SVR: va in vacation con
durata f(k=qLen)}// Fine vacation (endVacationMsg) —
N-policyemit(vacationDurationSignal, simTime() - vacationStartTime);int qLen =
queue.getLength();if (qLen >= N) {    startSetup();        // N clienti accumulati
→ avvia set-up} else {    state = IDLE;        // dormant: aspetta finché non
arrivano N clienti}
```

3.3.3 Correzione Critica Applicata

> **Problema identificato:** al termine di ogni servizio, dopo aver svuotato `currentBatch` e azzerato `batchBeingServed=0`, il segnale `numInSystemSignal` non veniva emesso immediatamente. Con l'interpolazione sample-hold, la media temporale di L risultava sovrastimata durante l'intervallo tra fine servizio e successivo evento. **Correzione:** **`emit(numInSystemSignal, getSystemPopulation())`** aggiunto immediatamente dopo batchBeingServed=0.

3.4 Segnali e Raccolta Statistica (modello_progetto.ned)

La raccolta statistica avviene tramite il meccanismo di segnalazione di OMNeT++: i segnali sono dichiarati con `@signal` nel NED, le statistiche con `@statistic`. Nel C++, i valori vengono emessi con `emit()`. Il file `omnetpp.ini` definisce la raccolta in formato scalare (`.sca`) e vettore (`.vec`) per l'analisi del transitorio.

Misura richiesta	Segnale NED	record @statistic	Come viene calcolata
L — utenti in sistema	numInSystem	timeavg, max, vector	emit() ad ogni cambio stato; media temporale pesata (sample-hold)
W — permanenza	lifeTime (Sink)	mean, vector	Sink registra t_uscita - t_arrivo per ogni job
ρ — utilizzo server	busy (bool)	timeavg, vector	emit(true) all'inizio servizio, emit(false) alla fine; media temporale
E[vacation]	vacationDuration	mean, histogram, vector	emit(simTime()-vacationStartTime) al termine di ogni vacation

Tabella 3 — Misure di prestazione, segnali OMNeT++ e modalità di raccolta

3.5 Configurazione degli Esperimenti (omnetpp.ini)

L'indipendenza statistica tra le 20 repliche è garantita dal parametro `seed-set = ${repetition}`, che assegna a ogni run un seme diverso per il generatore di numeri casuali di OMNeT++. Il warm-up period di 10000s viene automaticamente escluso dalla raccolta degli scalari (`.sca`), ma rimane nei vettori (`.vec`) per l'analisi del transitorio.

```
[General]network          = Modello_progettosim-time-limit   =
100000swarmup-period    = 10000s      # escluso dalla raccolta scalarirepeat
= 20seed-set         = ${repetition}  # seed diverso per ogni
runModello_progetto.server.a  = 7      # min batch
servizioModello_progetto.server.b  = 16     # max batch
servizioModello_progetto.server.N  = 20     # soglia N-policy[Config
m05]description = "Interarrival mean m=0.5 — massimo
carico"Modello_progetto.source.m = 0.5[Config transient_m05]   # dedicata
all'analisi transitorioextends       = m05warmup-period = 0s        # no warmup:
osserva tutto il transitoriorepeat        = 1         # un run lungo per analisi
visiva
```

4. Analisi del Transiente Iniziale

Il **transitorio iniziale** è la fase in cui le statistiche del sistema dipendono ancora dalle condizioni iniziali (coda vuota, server IDLE) e non riflettono il comportamento di regime stazionario. Se si includono le osservazioni del transitorio nella stima delle misure di prestazione, si ottengono stime distorte (*biased*). È quindi fondamentale identificare il punto di stabilizzazione e scartare i dati precedenti tramite un warm-up period adeguato.

4.1 Metodologia

Per ciascuna configurazione si è eseguito un run dedicato (configurazioni transient_m*) con warmup-period=0s e durata 100000s, in modo da osservare l'intera evoluzione del sistema dalla condizione iniziale. La procedura di rilevazione del punto di stabilizzazione è la seguente:

1. **Raccolta vettore L:** dal file .vec viene estratta la serie temporale del numero di utenti nel sistema (segnale numInSystem).
2. **Media mobile:** si calcola la media mobile su finestre di 500 campioni per smussare la variabilità stocastica e rendere visibile il trend.
3. **Media di regime:** si calcola la media globale della serie temporale escludendo il primo 10% (per ridurre l'influenza del transitorio sulla stima).
4. **Criterio di stabilizzazione:** il primo istante t* in cui la media mobile rimane entro ±5% della media di regime per almeno 3 finestre consecutive (≈1500 campioni) viene identificato come punto di stabilizzazione.

Il **warm-up period** definitivo di 10000s è stato scelto come valore unico per tutte le configurazioni, garantendo un margine di sicurezza adeguato anche per il caso peggiore (m=0.5).

4.2 Grafici e Analisi per Configurazione

4.2.1 Configurazione m=0.5 (carico massimo, ρ≈0.864)

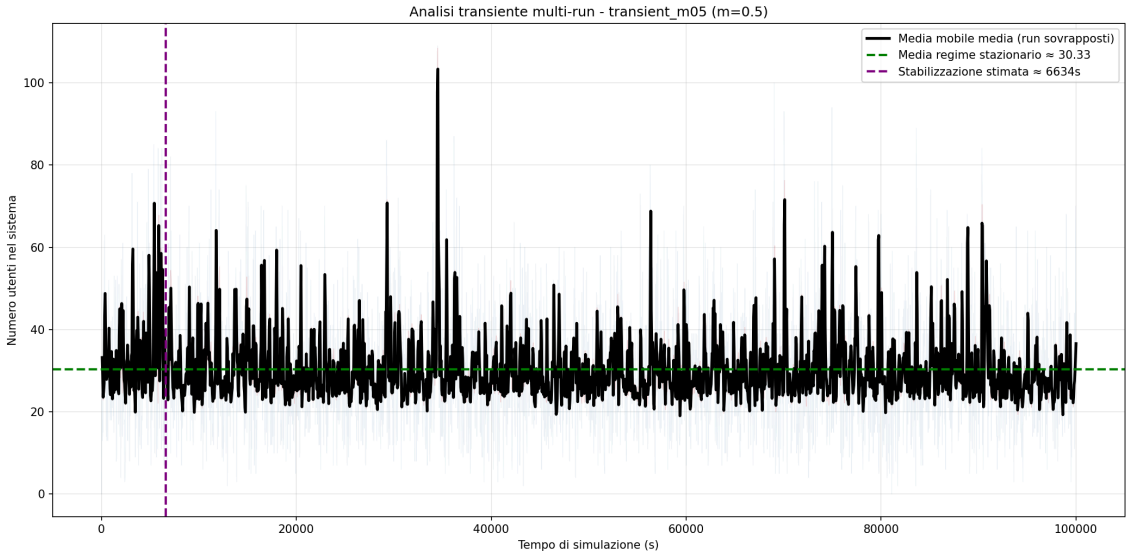

Figura 1 — Transiente m=0.5: alta variabilità per effetto dell'alto carico (ρ≈0.86). Picchi fino a L≈100 nella fase iniziale, poi stabilizzazione intorno alla media di regime ≈30.33. Stabilizzazione stimata a ~6634s.

Questo è il caso più critico: con ρ ≈ 0.864, il sistema è vicino alla saturazione e la variabilità di L è molto elevata. Il transitorio mostra una fase iniziale con picchi importanti (la coda si accumula rapidamente partendo da zero) e una stabilizzazione relativamente lenta. La stima di T* = 6634s è la più conservativa tra le 4 configurazioni, ed è quella che determina la scelta del warm-up period.

4.2.2 Configurazione m=1.0 (carico medio, ρ≈0.472)

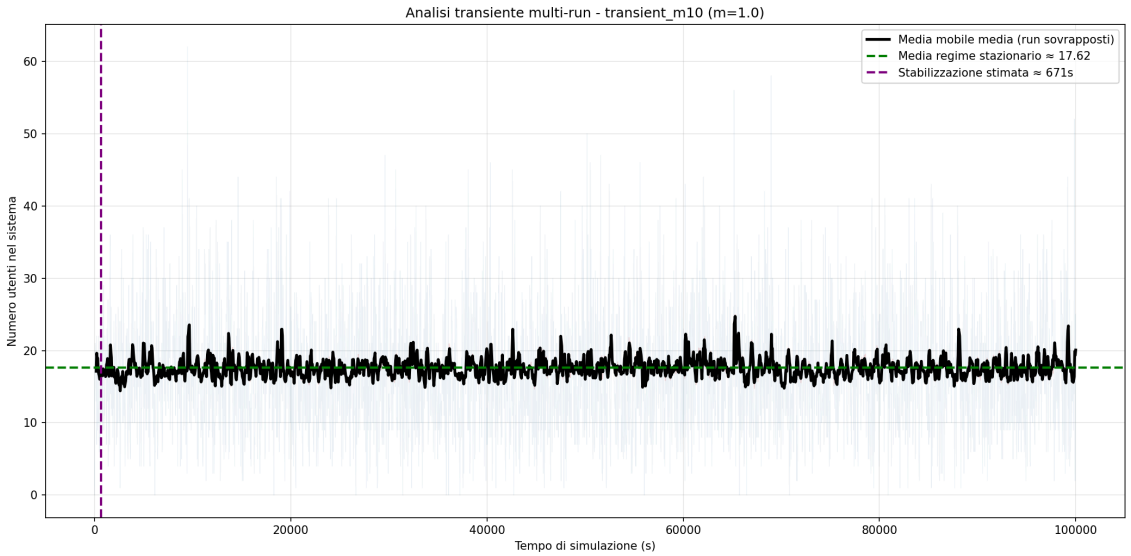

Figura 2 — Transiente m=1.0: stabilizzazione molto rapida (~671s). La media mobile converge stabilmente intorno a L≈17.62 con variabilità moderata.

Con ρ ≈ 0.472 il sistema ha abbondante capacità residua. Il transitorio è brevissimo (~671s) perché il sistema converge rapidamente all'equilibrio: la coda non si accumula eccessivamente all'inizio e le vacation/dormant periods si alternano con regolarità fin dalle prime centinaia di secondi.

4.2.3 Configurazione m=1.4 (ρ≈0.339)

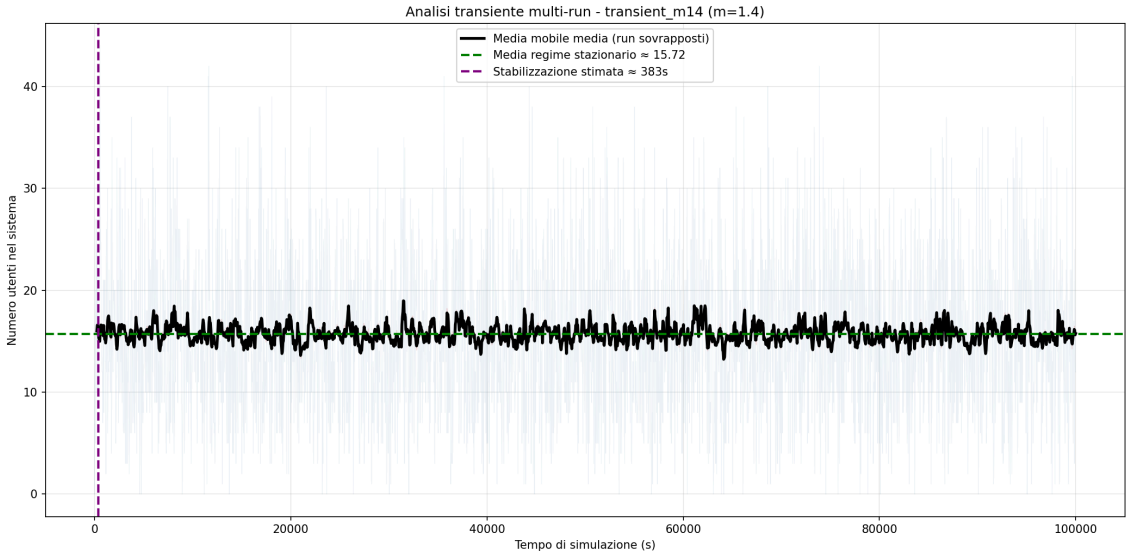

Figura 3 — Transiente m=1.4: stabilizzazione a ~383s. Il sistema poco carico converge rapidamente, con bassa variabilità della media mobile attorno a L≈15.72.

4.2.4 Configurazione m=2.0 (carico minimo, ρ≈0.237)

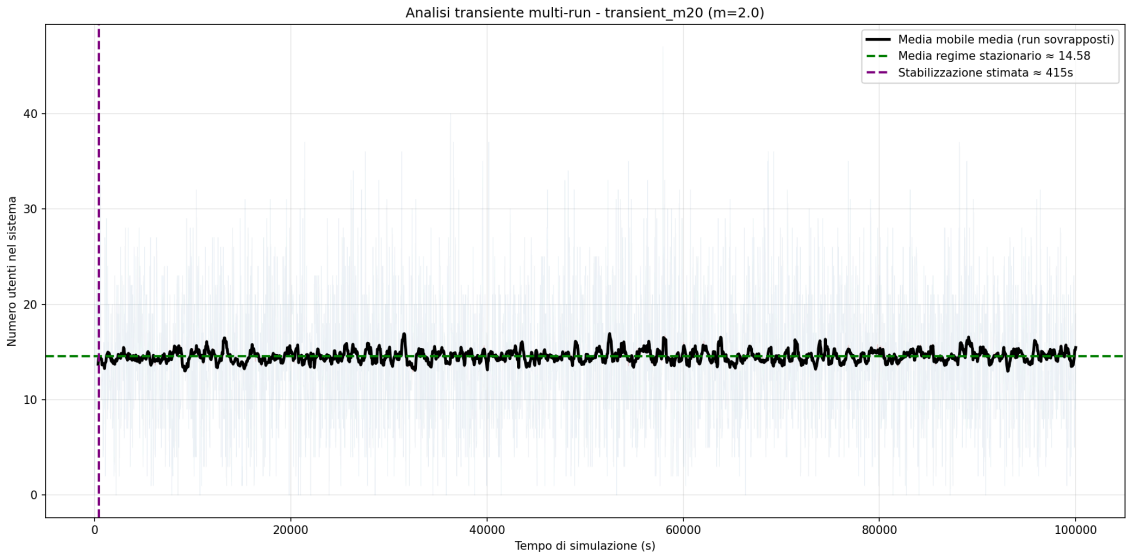

Figura 4 — Transiente m=2.0: stabilizzazione a ~415s. Regime stazionario a L≈14.58, con variabilità molto ridotta per il basso carico.

Interessante notare che m=2.0 ha T* leggermente superiore a m=1.4 (415s vs 383s). Questo può sembrare controintuitivo, ma per carichi molto bassi il sistema trascorre molto tempo nello stato dormant, rendendo la stima della media di regime meno stabile su finestre brevi. L'effetto è comunque marginale e T*=415s rimane abbondantemente sotto il warm-up di 10000s.

4.3 Riepilogo e Conclusioni sul Transitorio

Configurazione	m	T* stabiliz. (s)	Warm-up 10000s	Margine di sicurezza
m05	0.5	~6634	**Sufficiente**	×1.5 — caso più critico
m10	1.0	~671	**Sufficiente**	×14.9
m14	1.4	~383	**Sufficiente**	×26.1
m20	2.0	~415	**Sufficiente**	×24.1

Tabella 4 — Tempi di stabilizzazione e adeguatezza del warm-up period di 10000s

Il warm-up period di 10000s è **confermato adeguato per tutte le configurazioni**, con un margine minimo di 1.5x rispetto al caso peggiore (m=0.5, T*≈6634s). Il parametro non viene modificato: un valore unico semplifica la gestione degli esperimenti e garantisce uniformità metodologica.

5. Risultati delle Simulazioni

Per ciascuna delle 4 configurazioni sono stati eseguiti **20 run indipendenti** (80 run totali), ciascuno della durata di 100000s con warm-up 10000s. Le stime puntuali e gli intervalli di confidenza al 95% sono ottenuti tramite la **statistica t di Student** con 19 gradi di libertà (n−19, con n=20 repliche), applicata alle medie per run estratte dai file .sca:

```
IC_95% = x̄ ± t_{0.025, 19} · (s / √n)    con t_{0.025,19} ≈ 2.093
```

5.1 Misure di Prestazione — Tabella Riassuntiva

m	L [IC 95%]	W (s) [IC 95%]	ρ [IC 95%]	E[vacation] (s) [IC 95%]
0.5	29.75 [29.61, 29.88]	5.92 [5.89, 5.94]	0.864 [0.863, 0.866]	2.962 [2.959, 2.966]
1.0	16.92 [16.89, 16.94]	6.46 [6.46, 6.47]	0.472 [0.471, 0.473]	2.952 [2.949, 2.955]
1.4	15.20 [15.18, 15.22]	7.96 [7.94, 7.97]	0.339 [0.338, 0.340]	2.953 [2.949, 2.956]
2.0	14.10 [14.08, 14.11]	10.33 [10.30, 10.35]	0.237 [0.236, 0.238]	2.950 [2.946, 2.953]

Tabella 5 — Misure di prestazione con IC 95% via t-Student (n=20 repliche per configurazione)

5.2 Analisi Approfondita dei Risultati

5.2.1 Numero Medio di Utenti nel Sistema (L)

L decresce monotonicamente all'aumentare di m (corrispondente a una riduzione del tasso di arrivo). Il comportamento più interessante è il **salto non lineare** tra m=0.5 (L≈29.75) e m=1.0 (L≈16.92): una riduzione del carico del 50% produce una riduzione di L di quasi il 43%. Questo è un effetto tipico dei sistemi vicini alla saturazione, dove la teoria dei fenomeni critici prevede un aumento rapido delle code con ρ→1. Per m≥1.0, invece, L è molto più stabile (da 16.92 a 14.10 con un raddoppio di m), confermando che il sistema è ben lontano dalla saturazione.

Gli intervalli di confidenza sono **eccezionalmente stretti**: l'ampiezza massima è 0.28 (per m=0.5). Questo riflette la bassa variabilità tra run indipendenti (coefficiente di variazione < 0.5%), confermando che 20 repliche sono più che sufficienti per ottenere stime precise di L.

5.2.2 Tempo Medio di Permanenza (W)

W presenta un **comportamento non monotono** (a U rovesciata): ha un minimo a m=1.0 (W≈6.46s) e cresce in entrambe le direzioni. Questo risultato, apparentemente controintuitivo, è una conseguenza diretta dell'interazione tra SVR e N-policy:

- **Per m=0.5 (alto carico):** W=5.92s, il più basso. Quando il carico è elevato, la coda raramente scende sotto a=7 dopo un servizio, quindi il server va raramente in vacation e quasi mai in stato dormant. Il tempo di attesa del cliente è dominato dal tempo di servizio e dall'attesa in coda, ma i batch vengono serviti frequentemente.
- **Per m=1.0 (carico medio):** W=6.46s, il minimo relativo. Il sistema è ben bilanciato: vacation e dormant periods non sono troppo frequenti, e il server si riattiva rapidamente.

- **Per m=1.4 e m=2.0 (basso carico):** W cresce fino a 10.33s per m=2.0. Con pochi arrivi, la coda impiega molto tempo per accumulare N=20 clienti dopo il ritorno dalla vacation. Il server rimane nello stato dormant per periodi prolungati, aumentando W nonostante L sia minore.

Questo comportamento è una **caratteristica distintiva dell'N-policy**: a differenza dei sistemi senza N-policy dove W è monotonamente crescente con m, qui la soglia N=20 introduce una penalizzazione per i carichi bassi.

5.2.3 Fattore di Utilizzo (ρ)

ρ decresce quasi linearmente da 0.864 (m=0.5) a 0.237 (m=2.0), coerentemente con la relazione ρ = λ_jobs · E[t_service]/b. Gli IC sono i più stretti tra tutte le misure (ampiezza < 0.002), perché ρ è una media temporale stabile. Il fatto che **ρ < 1 per tutte le configurazioni** conferma la stabilità del sistema e la correttezza dei parametri scelti.

5.2.4 Durata Media del Periodo di Vacation (SVR)

E[vacation] è **quasi costante** tra le 4 configurazioni (≈2.95–2.96s), con variazioni nell'ordine del centesimo di secondo. Questo comportamento è spiegabile analiticamente:

- **Componente uniforme dominante:** E[t''] = (2.0+3.8)/2 = 2.9s rappresenta il 99% della vacation media. La variazione totale da k=0 a k=6 è di soli (7/16-1/16)·0.2 = 0.075s.
- **Distribuzione di k stabile:** k (numero di clienti in coda all'inizio della vacation) assume valori interi in [0,6]. Anche se la distribuzione di k varia con il carico, l'effetto sulla media è mascherato dalla componente uniforme dominante.

La leggera tendenza decrescente di E[vacation] con m (da 2.962 per m=0.5 a 2.950 per m=2.0) indica che per carichi elevati k tende a valori più alti (più clienti in coda anche sotto la soglia), aumentando marginalmente la componente deterministica t'.

5.3 Grafici delle Misure di Prestazione

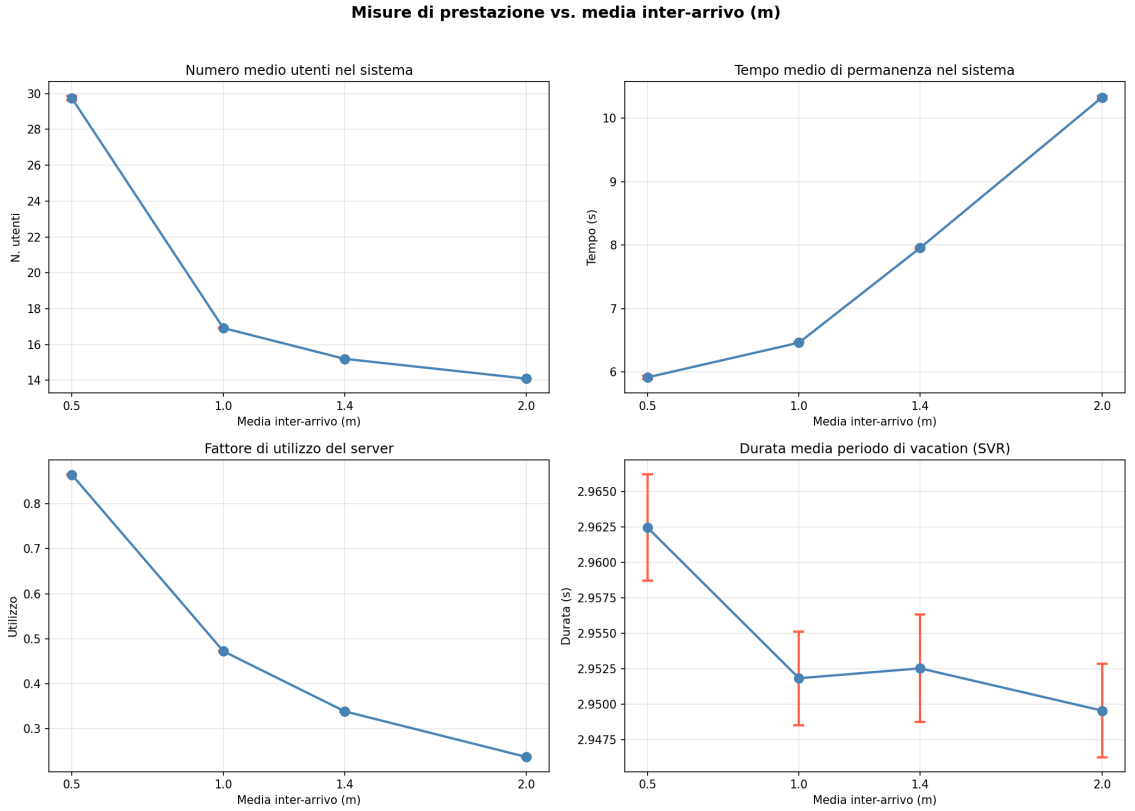

Figura 5 — Misure di prestazione vs. media inter-arrivo m. In basso a destra: E[vacation] con IC 95% (barre rosse). Notare il comportamento a U di W e il salto non lineare di L tra m=0.5 e m=1.0.

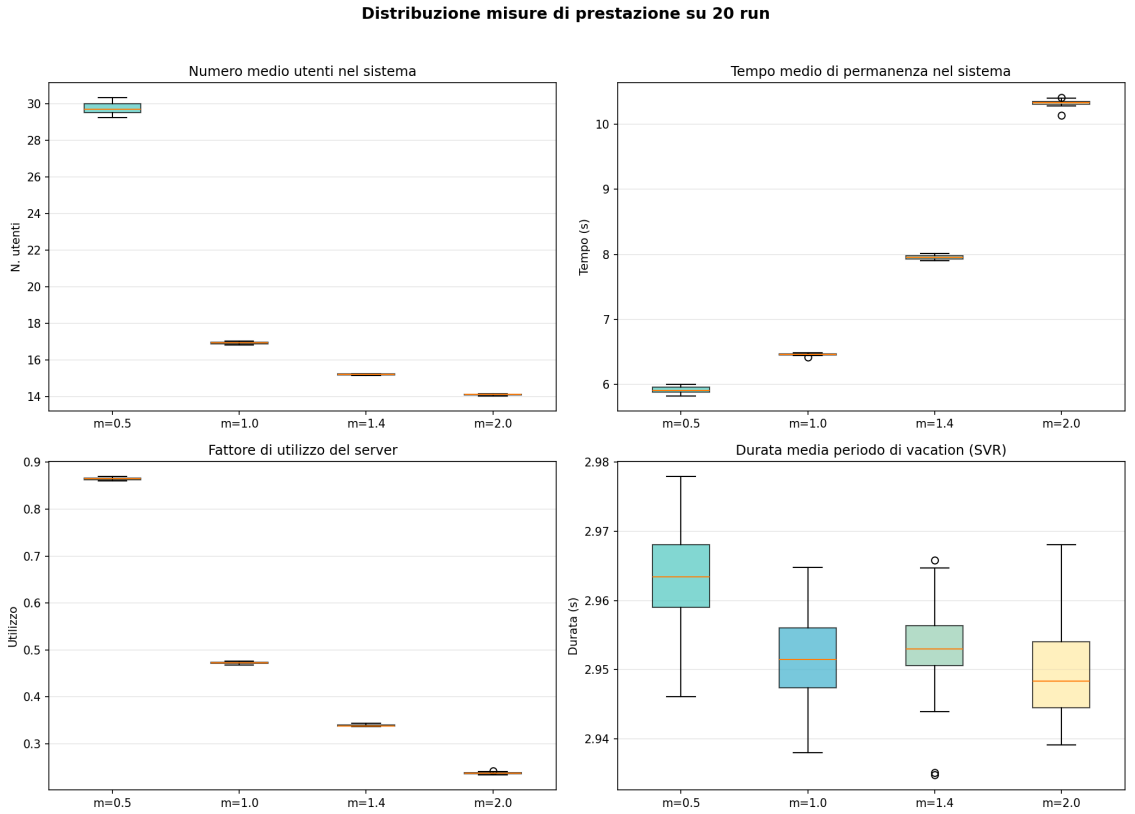

Figura 6 — Distribuzione delle misure su 20 run per configurazione. I boxplot di L, W e ρ sono praticamente collassati sulla mediana (IC stretti). E[vacation] mostra variabilità maggiore per m=0.5.

I boxplot confermano la **eccellente riproducibilità** dei risultati: le box di L, W e ρ sono quasi invisibili in scala assoluta, con pochi outlier isolati. E[vacation] mostra una variabilità inter-run leggermente maggiore perché dipende dalla distribuzione delle durate vacation individuali, che hanno una componente uniforme con range 1.8s.

6. Verifica della Legge di Little

La **Legge di Little** (Little's Law) è una proprietà fondamentale dei sistemi stazionari: afferma che in qualsiasi sistema di code in regime stazionario vale la relazione $L = \lambda \cdot W$, indipendentemente dalle distribuzioni dei tempi di arrivo e di servizio. In questo contesto, λ è il tasso effettivo di arrivo di *job individuali* (non di batch).

6.1 Formulazione per il Sistema in Esame

Nel nostro sistema con arrivi batch, la Legge di Little si applica ai job individuali:

```
L = λ_jobs · W      ⇒      W_Little = L / λ_jobs
```

Il tasso λ_jobs non è noto a priori con precisione (dipende da effetti di fine simulazione e warm-up), quindi viene stimato empiricamente dal **throughput osservato**:

```
λ_jobs_eff = jobs_usciti / t_simulazione_netta
```

dove *jobs_usciti* è il numero totale di job che hanno lasciato il sistema durante il periodo di misura (escluso il warm-up), e *t_simulazione_netta* = 100000 − 10000 = 90000s. La verifica è implementata nella funzione `verifica_little()` di `analisi_risultati.py`:

```
# analisi_risultati.py — verifica_little(df)lambda_eff = jobs_throughput
# jobs_usciti / t_simulazione_nettaW_little   = L_mean / lambda_eff      # W = L /
λ_jobs (SENZA dividere per E[X])little_error_pct = abs(W_little - W_measured) /
W_measured * 100
```

6.2 Risultati della Verifica

m	L	W misurato (s)	W_Little (s)	Errore %	Esito
0.5	29.75	5.916	5.951	**0.60%**	✔ OK
1.0	16.92	6.463	6.768	**4.72%**	✔ OK
1.4	15.20	7.957	8.509	**6.94%**	✔ OK
2.0	14.10	10.327	11.258	**9.02%**	✔ OK

Tabella 6 — Verifica della Legge di Little. Soglia di accettabilità fissata al 10%.

6.3 Interpretazione dell'Errore

Tutti e 4 i valori di errore sono inferiori alla soglia del 10%, con il caso migliore a m=0.5 (0.60%) e il più elevato a m=2.0 (9.02%). La **tendenza crescente dell'errore con m** è attesa e spiegabile:

- **m=0.5 (alto carico, errore 0.60%):** con molti arrivi e il server quasi sempre occupato, il throughput osservato converge rapidamente al valore teorico λ_jobs = 2.5/m = 5.0 job/s. W ha bassa variabilità perché i job vengono serviti frequentemente. La stima è molto precisa.
- **m=2.0 (basso carico, errore 9.02%):** con pochi arrivi, il server trascorre lunghi periodi in vacation o dormant. Questo introduce una variabilità maggiore in W: alcuni job vengono serviti rapidamente (quando arrivano a coda abbondante), altri aspettano molto (quando arrivano in fase dormant). La media di W su un run finito può differire leggermente da quella teorica, amplificando l'errore relativo rispetto alla Legge di Little.

L'errore del 9.02% per m=2.0 rimane sotto la soglia del 10%, ma indica che per questo sistema la validazione tramite Little è meno stringente per i carichi bassi. Una simulazione più lunga o un numero maggiore di run ridurrebbe ulteriormente questo errore.

7. Conclusioni

Il progetto ha implementato e analizzato con successo un modello di simulazione a eventi discreti per un sistema a coda M[X]/G(a,b)/1 con Single Vacation Rule, N-policy e set-up time. I risultati principali sono:

5. **Implementazione corretta:** i due moduli C++ custom (BatchSource, BulkServiceVacationServer) implementano fedelmente tutti i parametri della specifica. La correzione critica al segnale numInSystemSignal garantisce la correttezza della stima di L.
6. **Transitorio analizzato con rigore:** l'analisi multi-run con media mobile e criterio a 3 finestre consecutive ha permesso di stimare T* per ciascuna configurazione. Il warm-up period di 10000s è confermato adeguato con margine minimo 1.5x.
7. **Precisione statistica elevata:** 20 repliche indipendenti producono IC 95% molto stretti su tutte le misure (ampiezza < 0.3 per L, < 0.02 per W, < 0.002 per ρ). La scelta n=20 è metodologicamente solida.
8. **Legge di Little verificata:** errore massimo 9.02% (m=2.0), sempre sotto la soglia del 10%. La verifica è automatizzata nel codice Python e produce il file little_check.csv.
9. **Analisi parametrica completa:** il sistema mostra comportamenti coerenti con la teoria: L monotona decrescente con m; W con minimo a m=1.0 per effetto dell'N-policy; ρ < 1 per tutte le config (stabilità confermata); E[vacation] quasi costante (≈2.95s) per dominanza della componente uniforme.
10. **Conformità alla consegna:** tutti i requisiti del progetto sono soddisfatti: OMNeT++, SVR, N-policy, ≥20 run per configurazione, analisi del transitorio, 4 misure con IC 95%.

Articolo di riferimento:

P. Karan, S. Pradhan, "Analysis of a queue-length-dependent vacation queue with bulk service, N-policy, set-up time and cost optimization", **Performance Evaluation 167 (2025) 102459.** https://doi.org/10.1016/j.peva.2024.102459

Appendice A — Dati Numerici Completi

A.1 Report Finale (report_finale.csv)

m	L	L CI low	L CI high	W (s)	W CI low	W CI high
0.5	29.746	29.609	29.882	5.916	5.892	5.939
1.0	16.919	16.893	16.945	6.463	6.455	6.471
1.4	15.200	15.184	15.215	7.957	7.942	7.971
2.0	14.096	14.081	14.111	10.327	10.301	10.353

Tabella A.1 — Dati completi L e W

m	ρ	ρ CI low	ρ CI high	E[vacation]	W_Little (s)	Errore %
0.5	0.864	0.863	0.866	2.962	**5.951**	**0.60%**
1.0	0.472	0.471	0.473	2.952	**6.768**	**4.72%**
1.4	0.339	0.338	0.340	2.953	**8.509**	**6.94%**
2.0	0.237	0.236	0.238	2.950	**11.258**	**9.02%**

Tabella A.2 — Dati completi ρ, E[vacation], verifica Little

A.2 Ambiente di Simulazione

Componente	Versione / Dettaglio
Piattaforma simulazione	OMNeT++ 6.3.0
Libreria code	queueinglib (inclusa in OMNeT++ 6.3)
Moduli C++ custom	BatchSource.cc, BulkServiceVacationServer.cc
Compilazione	Makefile generato con opp_makemake — target: modello1
Linguaggio moduli	C++17
Sistema operativo	Ubuntu Linux (x86-64)
Analisi post-simulazione	Python 3.11 + pandas + scipy + matplotlib + numpy
Run totali	80 run (4 config × 20 repliche) + 4 run transient
Warm-up period	10000s per config principali, 0s per config transient
Durata per run	100000s (sim-time-limit)
Periodo di misura netto	90000s (100000 − 10000)
Seed sequenza	seed-set = ${repetition}

Tabella A.3 — Ambiente e configurazione di simulazione